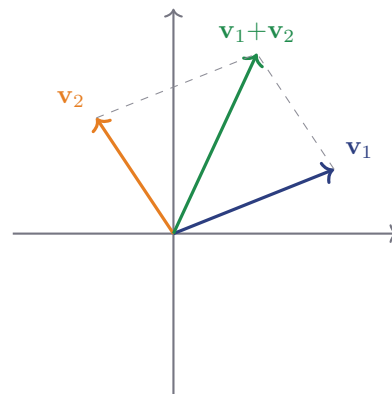


Vector Spaces

The Architecture of Machine Learning Math

Lecture 2 • Groups • Subspaces • Independence • Basis • Dimension



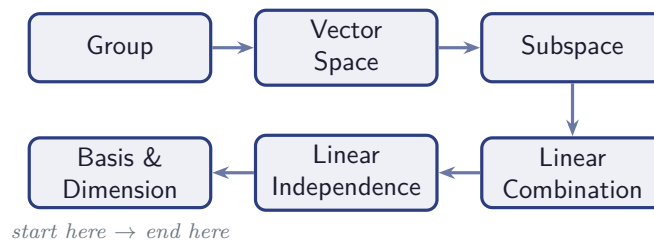
“We stop looking at individual equations and start looking at the universe where these mathematical objects live.”

What you will gain from this companion:

- A guided, visual tour through every concept in Lecture 2 — built around *pictures, intuition, and ML use-cases* rather than dense definitions.
- Worked examples for the trickiest results, especially the *linear-independence-of-combinations* theorem.
- Clear ML/DS framing for every concept: what it *is*, why we *care*, and where it *shows up* in real models.

Contents

1	Why Vector Spaces? An ML-First Motivation	3
2	Groups: The Bare Skeleton of “Combining Things”	4
3	Vector Spaces: Group + Scaling	7
4	Subspaces: Spaces Inside Spaces	10
5	Linear Combinations and Span: What You Can Reach	12
6	Linear Independence: The Art of Non-Redundancy	14
7	The Independence-of-Combinations Theorem	17
8	Basis and Dimension: The Skeleton of a Space	20
9	Finding a Basis: The Algorithm	23
10	Putting It All Together	25
11	Practice Problems	26



This companion follows that journey. Each section has: *the formal idea*, *visual intuition*, *worked examples*, and *the ML perspective*.

1. Why Vector Spaces? An ML-First Motivation

In Lecture 1 we played with systems of equations $A\mathbf{x} = \mathbf{b}$. We found solutions, talked about pivots, manipulated rows. That was *tactical*. Lecture 2 is *strategic*. We zoom out and ask: **what kind of world do these vectors live in?**

The answer is: a structured world we call a *vector space*. Once you know the rules of this world, every algorithm in machine learning starts making sense as a small geometric story inside it.

Reading the notation behind every ML object

Before we go any further, let's decode the symbols you'll see throughout MFML. They look intimidating, but each one is just a way of saying "a list of real numbers with a particular shape."

(a) $\mathbb{R}$ — the set of all real numbers (anything you can put on a number line: 0, -3.14 , $\sqrt{2}$, 42, ...). A single real number is sometimes called a *scalar*.

(b) $\mathbb{R}^n$ — a list (or column) of n real numbers. The exponent n tells you *how many* numbers. Example:

$$\mathbb{R}^3 = \left\{ (x_1, x_2, x_3) : x_i \in \mathbb{R} \right\}, \quad \text{e.g., } \begin{bmatrix} 2.1 \\ -3 \\ 0.7 \end{bmatrix} \in \mathbb{R}^3.$$

So $\mathbb{R}^{768}$ just means "a column with 768 real numbers in it." That's it.

(c) $\mathbb{R}^d$ or $\mathbb{R}^k$ — same as $\mathbb{R}^n$, but using a placeholder letter (d, k, n) when we don't care about the specific number. "A vector in $\mathbb{R}^d$ " just means "a vector with *some* number of components, called d ."

(d) $\mathbb{R}^{m \times n}$ — the set of all m -row, n -column matrices of real numbers. m comes first (rows), n second (columns). Example:

$$\mathbb{R}^{2 \times 3} : \begin{bmatrix} 1 & 0 & -2 \\ 4 & 5 & 7 \end{bmatrix}.$$

A matrix in $\mathbb{R}^{m \times n}$ has $m \cdot n$ entries total.

(e) $\mathbb{R}^{H \times W \times 3}$ — a 3-dimensional "box" of numbers (a *tensor*) with H rows, W columns, and 3 layers. The classic example: a color image of height H pixels and width W pixels, where the 3 layers are the Red, Green, and Blue channels. So a 224×224 RGB photo is in $\mathbb{R}^{224 \times 224 \times 3}$, which is $224 \cdot 224 \cdot 3 = 150,528$ numbers.

The pattern. The numbers in the exponent (after $\mathbb{R}$) describe the *shape* of the data:

Notation	Shape	Mental picture
$\mathbb{R}$	a single number	a dot on a number line
$\mathbb{R}^n$	a column of n numbers	a vector / arrow
$\mathbb{R}^{m \times n}$	an $m \times n$ grid	a spreadsheet table
$\mathbb{R}^{H \times W \times 3}$	a stack of 3 grids	an RGB image
$\mathbb{R}^{B \times H \times W \times 3}$	B such images	a batch of images

Read each $\mathbb{R}^{(\text{shape})}$ as "the set of all real-valued data with this shape."

Why an ML student should care

Almost every object in ML is a vector in some vector space:

- A **photograph** of a cat is a vector in $\mathbb{R}^{H \times W \times 3}$ — i.e., a 3-D tensor with height H , width W , and 3 color channels (R, G, B).
- A **sentence** “the food was great” is a sequence of word-embedding vectors in $\mathbb{R}^{768}$ (i.e., each word becomes a column of 768 real numbers, the BERT-base size) or $\mathbb{R}^{4096}$ (the typical hidden size for many modern LLMs).
- A **user** on Netflix is a vector of preferences in some $\mathbb{R}^d$ — one column of d numbers, where the choice of d depends on the recommender system.
- A **neural network’s weights** are matrices in $\mathbb{R}^{m \times n}$: an m -row, n -column grid of real numbers, which is itself a “vector” once we view the whole grid as a single object.

Training a model is essentially about *moving* inside these spaces in clever ways. If you don’t know the rules of the space, you can’t reason about the move.

The big-picture analogy

Think of a vector space as the **board game** ML plays on. Groups give us the *addition rules*. Vector spaces add *scaling rules*. Subspaces are valid *regions* of the board. Linear independence says *which moves are redundant*. A basis is the *minimal toolkit* that can reach every square. Dimension is *how big the board really is*. Master those six pieces, and the rest of MFML is mostly geometry.

2. Groups: The Bare Skeleton of “Combining Things”

Before we can define a vector space, we need a more primitive structure: a *group*. A group is the minimum amount of structure we need so that “combining two things gives us a sensible third thing.”

Group — in plain English

A **group** is just a *set of things* (call it G) together with a way to *combine any two of them* (call this combine-operation $\otimes$). For $(G, \otimes)$ to deserve the name “group,” the combine-operation has to behave nicely in four ways:

- (G1) **Closure** — combining two elements of G gives back something in G . (*“You don’t fall out of the set.”*)
- (G2) **Associativity** — the order in which you group your operations doesn’t matter: $(a \otimes b) \otimes c$ equals $a \otimes (b \otimes c)$. (*Think $(1+2)+3 = 1+(2+3)$.*)
- (G3) **Identity** — there’s a special “do-nothing” element e in the set: combining with e leaves any element unchanged. (*For addition, $e = 0$. For multiplication, $e = 1$.*)
- (G4) **Inverse** — every element has a partner that “undoes” it, returning you to e . (*For addition, the partner of 5 is -5 .*)

Bonus property: If the order of combination doesn’t matter either ($a \otimes b = b \otimes a$), we call the group **Abelian**. (Vanilla addition is Abelian; matrix multiplication is not.)

2.1 The four axioms, visually

Closure Stay inside the set $x+y$ stays an integer	Associativity Grouping doesn't matter $(1+2)+3 = 1+(2+3)$	Identity “Do nothing” element $x+0 = x$	Inverse Everything is reversible $x+(-x)=0$
---	--	--	--

2.2 A tour of examples (in plain English)

Let’s walk through five candidates and check, in everyday terms, whether each is a group. Before we start, a quick note on the symbol shorthands you’ll see used throughout MFML:

Number-set shorthands you’ll see in MFML

Mathematicians have “shorthand” letters for popular sets of numbers. Here are the ones we use:

Symbol	What it means (plain English)	example members
$\mathbb{N}$	The natural numbers (counting numbers $1, 2, 3, \dots$)	$1, 7, 42$
$\mathbb{N}_0$	The natural numbers <i>including</i> zero	$0, 1, 7, 42$
$\mathbb{Z}$	The integers — whole numbers, positive and negative	$-3, 0, 7$
$\mathbb{R}$	The real numbers — anything on the number line	$-2.7, 0, \sqrt{2}, 42$
$\mathbb{R} \setminus \{0\}$	The real numbers <i>except</i> zero	$-2.7, 1, 42$ (but not 0)
$\mathbb{R}^n$	Columns of n real numbers (vectors)	$\begin{bmatrix} 1 \\ -3 \end{bmatrix} \in \mathbb{R}^2$

The notation $(S, \diamond)$ just means “the set S paired with the operation $\diamond$.” For example, $(\mathbb{Z}, +)$ is read as “the integers, with ordinary addition.”

Example A — All whole numbers (positive and negative) with addition

Set + op: $(\mathbb{Z}, +)$. So our elements are $\dots, -2, -1, 0, 1, 2, \dots$ and we combine using $+$.

- Adding two whole numbers gives a whole number. **Closure** ✓
- Grouping doesn’t matter: $(2+3)+5 = 2+(3+5)$. **Associativity** ✓
- Adding 0 leaves any number unchanged. **Identity** ✓ (the do-nothing element is 0)
- Every number n has a partner $-n$ that cancels it: $5 + (-5) = 0$. **Inverse** ✓

Verdict: Abelian group. This is the simplest, friendliest example to keep in mind.

Example B — Counting numbers (with zero) under addition

Set + op: $(\mathbb{N}_0, +)$. Our elements are $0, 1, 2, 3, \dots$ (no negatives allowed).

- Adding two counting numbers gives a counting number. **Closure** ✓
- Grouping doesn’t matter. **Associativity** ✓
- 0 still acts as the identity. **Identity** ✓
- *Here’s the problem:* how do you “undo” adding 5? You’d need to add -5 — but -5 isn’t in the set. There’s no way back to 0 once you’ve moved away from it. **Inverse** ✗

Verdict: Not a group — no inverses. This is the lesson: even one missing axiom disqualifies you.

Example C — All whole numbers under multiplication

Set + op: $(\mathbb{Z}, \cdot)$. Same elements as Example A, but now we multiply instead of add.

- Multiplying two whole numbers gives a whole number. **Closure** ✓
- Multiplication is associative. **Associativity** ✓
- Multiplying by 1 leaves any number unchanged. **Identity** ✓ (the do-nothing element is 1)
- *Now the problem:* how do you “undo” multiplying by 2? You’d divide by 2, i.e., multiply by $\frac{1}{2}$. But $\frac{1}{2}$ isn’t a whole number! Most whole numbers have no inverse here. **Inverse** ✗

Verdict: **Not a group**. Even though identity exists, almost no one has a partner.

Example D — All non-zero real numbers under multiplication

Set + op: $(\mathbb{R} \setminus \{0\}, \cdot)$. Take all the numbers on the number line, except 0, and multiply them.

- Two non-zero reals multiply to a non-zero real. **Closure** ✓
- Associativity is fine. **Associativity** ✓
- 1 is the do-nothing. **Identity** ✓
- Every non-zero x has a partner $\frac{1}{x}$ (also a non-zero real), and $x \cdot \frac{1}{x} = 1$. **Inverse** ✓

Verdict: Abelian group. **Why we excluded zero:** 0 has no multiplicative inverse (you can’t divide by zero). Removing it fixes the inverse axiom.

Example E — Vectors of any fixed dimension under addition

Set + op: $(\mathbb{R}^n, +)$. Our elements are vectors (columns of n real numbers); we add them entry-by-entry.

- Vector + vector = vector (same shape). **Closure** ✓
- Adding entry-by-entry inherits associativity. **Associativity** ✓
- The all-zeros vector **0** does nothing when added. **Identity** ✓
- Every **x** has a partner $-\mathbf{x}$ that cancels it: $\mathbf{x} + (-\mathbf{x}) = \mathbf{0}$. **Inverse** ✓

Verdict: Abelian group. This one matters most for ML — almost every vector you’ll meet (gradients, embeddings, weight matrices) lives in this kind of group.

Summary at a glance

Plain-English example	Cl	As	Id	Inv	verdict
A. Whole numbers $\pm$ $(\mathbb{Z}, +)$	✓	✓	✓	✓	Abelian group
B. Counting numbers ≥ 0 $(\mathbb{N}_0, +)$	✓	✓	✓	✗	not a group (can’t undo)
C. Whole numbers, multiplied $(\mathbb{Z}, \cdot)$	✓	✓	✓	✗	not a group (no fractions)
D. Non-zero real numbers, multiplied $(\mathbb{R} \setminus \{0\}, \cdot)$	✓	✓	✓	✓	Abelian group
E. Vectors of any size, added $(\mathbb{R}^n, +)$	✓	✓	✓	✓	Abelian group

The pattern — what to look for

Closure and associativity rarely break; they're the easy ones. The two axioms that knock candidates out are:

- **Identity** — “Is there a do-nothing element?” (For addition that's 0; for multiplication that's 1.)
- **Inverse** — “Can I always undo what I did?” This one fails most often — whenever your set is missing the negatives, fractions, or whatever “undo” partner each element needs.

When checking any new structure, just walk these four questions in order. The moment one fails, you've found the issue.

Bonus real-world group — clock arithmetic

Imagine the hours on a 12-hour clock: $\{0, 1, 2, \dots, 11\}$ where “addition” means “advance the hour hand and wrap around past 12.” For example, $9 + 5 = 14$, but on the clock that's $14 - 12 = 2$. So $9 + 5 = 2$ in clock-world.

This *is* an Abelian group: closure (you stay on the clock), associativity (grouping doesn't matter), identity (0 does nothing), inverse (the partner of 5 is 7 because $5 + 7 = 12 \equiv 0$). It's the same four axioms, just on a finite, circular set — and it's the foundation of cryptography, hashing, and the math behind modern AI's positional embeddings.

ML Perspective

You won't typically write “let G be a group” in your training loop. But you rely on group structure constantly:

- Gradient descent does $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$. This is just *repeated addition in the group* $(\mathbb{R}^d, +)$. Closure guarantees the new weight vector still lives in $\mathbb{R}^d$.
- Rotations of an image (used in data augmentation) form a group $SO(2)$. Composing two rotations gives a rotation; every rotation is invertible.
- “Permutations of tokens” in a sentence form the symmetric group S_n — the rules attention layers must respect (or break, in the case of position encodings).

3. Vector Spaces: Group + Scaling

A vector space is what happens when you take an Abelian group and bolt on the ability to *stretch* elements by real numbers.

Vector Space — in plain English

A **vector space** is a set of objects (we call them *vectors*) where you're allowed to do exactly two things:

- **Add two vectors** and get another vector. (*“Inner” operation: $\mathbf{u} + \mathbf{v}$ stays in the space.*)
- **Scale a vector** by a real number and get another vector. (*“Outer” operation: $3\mathbf{u}$ or $-0.5\mathbf{u}$ stays in the space.*)

And these two operations must play well together:

- (V1) Vector addition is an **Abelian group** — i.e., addition is associative and commutative, there's a zero vector $\mathbf{0}$ that does nothing, and every $\mathbf{v}$ has a negative $-\mathbf{v}$ that cancels it.
- (V2) **Distributivity** — scaling distributes over addition, just like in school algebra: $\lambda(\mathbf{x} + \mathbf{y}) = \lambda\mathbf{x} + \lambda\mathbf{y}$ and $(\lambda + \psi)\mathbf{x} = \lambda\mathbf{x} + \psi\mathbf{x}$.
- (V3) **Scaling stacks** — scaling by λ and then by ψ is the same as scaling by $\lambda\psi$ once: $\lambda(\psi\mathbf{x}) = (\lambda\psi)\mathbf{x}$.
- (V4) **Scaling by 1 does nothing** — $1 \cdot \mathbf{x} = \mathbf{x}$.

Translation. A vector space is just a place where the two basic moves of linear algebra (add, scale) always make sense and never throw you out of the playground.

The drawing-board picture

Imagine an infinite drawing board with an origin marked $\mathbf{0}$. A vector is a *movement instruction* from the origin.

- **Addition** chains movements: “go along $\mathbf{u}$, then along $\mathbf{v}$.”
- **Scaling** stretches a movement: “go three times as far along $\mathbf{u}$.”

The vector-space axioms are the guarantees that no matter how you mix these moves, *you never fall off the board*. That closure is what makes the space “a space”.

3.1 Two essential examples (told from scratch)

Example 1 — “Lists of numbers” ($V = \mathbb{R}^n$). The simplest, most familiar vector space. A vector is just a column of n real numbers. Picture three students' exam scores: $(85, 92, 78)$ is a vector in $\mathbb{R}^3$. Adding two such vectors means adding them entry by entry:

$$\begin{bmatrix} 85 \\ 92 \\ 78 \end{bmatrix} + \begin{bmatrix} 5 \\ -2 \\ 3 \end{bmatrix} = \begin{bmatrix} 90 \\ 90 \\ 81 \end{bmatrix}.$$

“Doubling the bonus” (scaling by 2) just multiplies every entry by 2. The result of either operation is still a column of three real numbers — so we never leave $\mathbb{R}^3$. That's all the vector-space axioms ask for.

Example 2 — “Excel sheets” ($V = \mathbb{R}^{m \times n}$). *The vectors here are matrices.* Imagine a tiny 2×3 spreadsheet of weekly sales for two products across three regions. That *whole sheet* is one “vector” in $\mathbb{R}^{2 \times 3}$. Adding two sheets adds the corresponding cells; scaling a sheet by 2 doubles every cell. For instance:

$$\underbrace{\begin{bmatrix} 12 & 8 & 5 \\ 7 & 9 & 11 \end{bmatrix}}_{\text{Week 1 sales}} + \underbrace{\begin{bmatrix} 3 & 4 & -1 \\ 2 & -3 & 0 \end{bmatrix}}_{\text{Week 2 change}} = \underbrace{\begin{bmatrix} 15 & 12 & 4 \\ 9 & 6 & 11 \end{bmatrix}}_{\text{combined}}.$$

Same idea, slightly bigger object. **The big lesson:** “vector” is just the name we give to whatever a vector space contains — it doesn’t have to be a column. It can be a matrix, a polynomial, a function, anything — as long as the addition and scaling rules behave.

Don’t get attached to the word “vector”

The word “vector” in linear algebra does *not* mean “column of numbers.” It means *any element of a vector space*. A 4×4 matrix is a vector. A polynomial is a vector. A function $f : \mathbb{R} \rightarrow \mathbb{R}$ can be a vector. Same axioms, very different objects.

Where every ML student meets this

ML object	Lives in vector space
A 224×224 RGB image	$\mathbb{R}^{224 \times 224 \times 3}$
A BERT word embedding	$\mathbb{R}^{768}$
A weight matrix in a dense layer	$\mathbb{R}^{m \times n}$
A grayscale digit (MNIST)	$\mathbb{R}^{28 \times 28} \cong \mathbb{R}^{784}$
A probability distribution over K classes	a subset of $\mathbb{R}^K$

Why this matters: the closure axiom is the reason “add a small gradient” or “average two model weights” (used in federated learning) gives back a valid model. The space is closed under those moves.

4. Subspaces: Spaces Inside Spaces

A vector space sometimes contains smaller, self-contained vector spaces inside it. We call those *subspaces*.

Vector Subspace — in plain English

A **subspace** is just a smaller vector space *living inside* a bigger one. Concretely: a subset U of a vector space V that's still a self-contained vector space all by itself, using the same addition and scaling rules.

Think of it like a flat plane sitting inside the 3-D room you're in: the plane is its own little 2-D world that follows all the same rules as the surrounding 3-D space.

The good news: most of the axioms come for free, because U inherits them from V . We only need to verify three things:

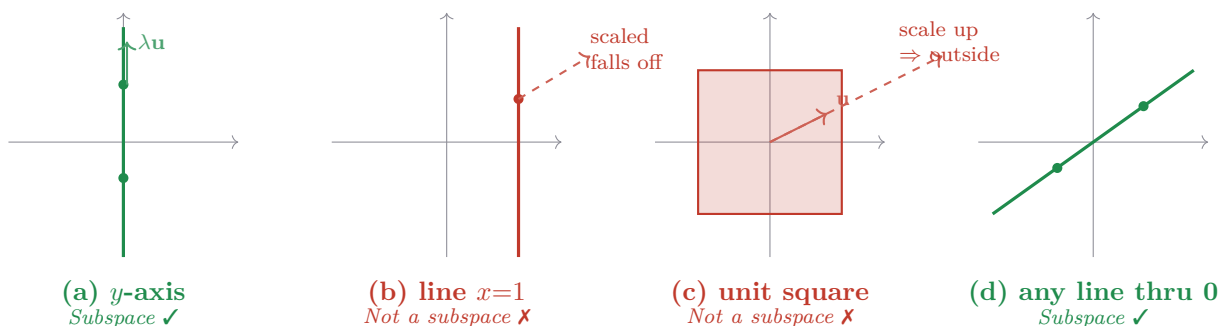
The Subspace Test — 3 things to check

A subset U of a vector space V is a subspace if (and only if) all three are true:

- (S1) **It contains the origin.** The zero vector $\mathbf{0}$ must be in U . (If $\mathbf{0}$ isn't in your set, stop — it can't be a subspace.)
- (S2) **It's closed under adding.** Pick any two vectors in U , add them: the result must *still* be in U .
- (S3) **It's closed under scaling.** Pick any vector in U and any real number λ : the scaled vector $\lambda\mathbf{u}$ must *still* be in U .

Why only three? Everything else (associativity, distributivity, etc.) is inherited automatically from the bigger space V , since U uses the same operations. We only need to verify that we don't accidentally leave U when we add or scale.

4.1 A visual gallery: subspace or not?



Walking through each panel:

- (a) **The y-axis.** Add any two points on this vertical line, the result is still on the line. Multiply any point by any number (positive, negative, zero), still on the line. Passes all three subspace tests. **Subspace.**
- (b) **Vertical line shifted to $x = 1$.** Pick a point on this line. Multiply it by 2 — the new point has $x = 2$, so it's *off* the line. Scaling kicks you out of the set. (Also: this line doesn't even pass through the origin.) **Not a subspace.**
- (c) **The unit square around the origin.** Take a point inside the square, like $\mathbf{u} = (0.8, 0.4)$. Scale it by 3: you get $(2.4, 1.2)$, which is outside the square. The set is bounded, so big scalings always escape. **Not a subspace.**

- (d) **Any line through the origin.** Same logic as the y -axis: scaling and adding stay on the line. **Subspace.**

The one-line rule of thumb

In $\mathbb{R}^n$, the only subspaces are flat shapes that pass through the origin: lines through $\mathbf{0}$, planes through $\mathbf{0}$, the whole space, and just $\{\mathbf{0}\}$ itself. Anything that doesn't pass through $\mathbf{0}$, or anything curved or bounded (like the square), is *not* a subspace — because somewhere along the way, addition or scaling will take you outside.

4.2 The most important subspace: the null space

Null Space of a Matrix — in plain English

Take a matrix A (m rows, n columns). Feed it different input vectors $\mathbf{x}$ and look at what comes out: $A\mathbf{x}$.

The **null space** of A is the set of *all the inputs that A flattens to zero*:

$$\text{null}(A) = \{ \mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{0} \}.$$

You can think of it as “the directions A can't see” — vectors that the matrix sends to nothing.

Why is this a subspace? Walk through the test:

1. It contains $\mathbf{0}$ since $A\mathbf{0} = \mathbf{0}$.
2. If $A\mathbf{x} = \mathbf{0}$ and $A\mathbf{y} = \mathbf{0}$, then $A(\mathbf{x} + \mathbf{y}) = A\mathbf{x} + A\mathbf{y} = \mathbf{0}$. Closed under addition.
3. If $A\mathbf{x} = \mathbf{0}$, then $A(\lambda\mathbf{x}) = \lambda(A\mathbf{x}) = \mathbf{0}$. Closed under scaling.

So $\text{null}(A)$ passes the subspace test for any matrix A .

Why null spaces matter for ML

The null space describes *the directions a linear map cannot see*. In ML this is everywhere:

- In a **linear regression** $\mathbf{y} = X\mathbf{w}$, any $\mathbf{w}$ in the null space of X leaves predictions unchanged. That's why we need regularization when X has a non-trivial null space (collinear features!).
- In **PCA**, the directions we throw away form a subspace; the kept directions form a complementary subspace. Both are subspaces.
- In **neural network compression**, “pruning a weight” is essentially saying “this direction is in the (effective) null space of the layer” so removing it costs little.

5. Linear Combinations and Span: What You Can Reach

Now we ask: *starting from a few vectors, where can scaling-and-adding take us?*

Linear Combination — in plain English

Take a few vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_k$ and a few real numbers $\lambda_1, \lambda_2, \dots, \lambda_k$ (called *weights* or *coefficients*). **Stretch each vector by its weight, then add them all up:**

$$\mathbf{v} = \lambda_1 \mathbf{x}_1 + \lambda_2 \mathbf{x}_2 + \dots + \lambda_k \mathbf{x}_k.$$

The result $\mathbf{v}$ is called a **linear combination** of those vectors. Different choices of weights give different combinations.

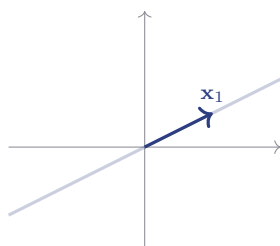
Span — in plain English

The **span** of a set of vectors is *everything you can possibly build* by linear combinations of them. Try every possible choice of weights $\lambda_1, \dots, \lambda_k$ — the collection of all the resulting vectors is the span:

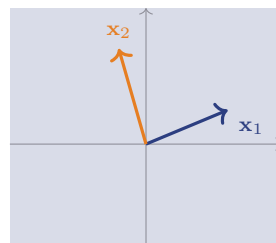
$$\text{span}(\mathbf{x}_1, \dots, \mathbf{x}_k) = \left\{ \lambda_1 \mathbf{x}_1 + \dots + \lambda_k \mathbf{x}_k : \lambda_i \in \mathbb{R} \right\}.$$

Important fact: the span is always a subspace — i.e., it always passes the three subspace tests.

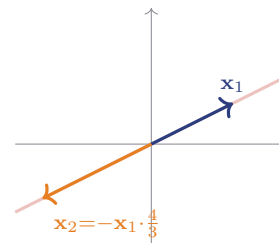
5.1 What span looks like, in pictures



1 vector
span = a line



2 indep vectors
span = a plane



2 collinear vectors
span still just a line

Walking through each panel:

- **Left — one vector.** You can scale $\mathbf{x}_1$ by any number, but every result lies on the same line through the origin. **Span = a line.**
- **Middle — two vectors pointing in different directions.** By mixing them with all possible weights you fill out an entire 2-D plane. **Span = a plane.**
- **Right — two vectors that point along the same line** (one is just a scaling of the other). No matter how you mix them, you stay on that single line — the second vector adds nothing new. **Span still = the same line.**

The third panel is the lesson worth tattooing: *adding a redundant vector doesn't grow the span.* That redundancy is exactly what *linear dependence* (next section) formalizes.

A concrete span calculation

Take just two specific vectors in $\mathbb{R}^2$: $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ (one step east) and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ (one step north). Mix them with any weights:

$$3\mathbf{e}_1 + 4\mathbf{e}_2 = \begin{bmatrix} 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 4 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

By picking the right weights, you can hit *any* point (a, b) : just take $a\mathbf{e}_1 + b\mathbf{e}_2$. So the span of these two vectors is the *entire 2-D plane*. That's why $\{\mathbf{e}_1, \mathbf{e}_2\}$ is the standard “coordinate basis” for $\mathbb{R}^2$.

Span as “reachable destinations”

Picture each vector as a transport mode: a train going north-east, a bus going south, a bike going east. The span is *every place on the map you can possibly arrive at* by combining these modes (walking backwards counts — that's a negative weight). If two of your modes go the same direction, the second one didn't actually expand your reach: the map of reachable destinations stays the same.

Span in machine learning

- **Word embeddings.** Take the vectors for **king**, **queen**, **man**, **woman**. Their span is a small “concept subspace” inside the giant embedding space. Famous demonstrations like **king** − **man** + **woman** $\approx$ **queen** live entirely in this span.
- **Linear regression.** The hypothesis class $\hat{\mathbf{y}} = X\mathbf{w}$ literally outputs vectors in $\text{span}(\text{columns of } X)$ — also called the *column space* of X .
- **Low-rank approximation & PCA.** Compressing data means projecting it onto the span of the top few eigenvectors. The smaller the span, the more you've compressed.

6. Linear Independence: The Art of Non-Redundancy

Linear (In)dependence — in plain English

The question: can you combine the vectors $\mathbf{x}_1, \dots, \mathbf{x}_k$ with weights and have them *cancel each other out* to give the zero vector — without setting all the weights to zero?

Linearly dependent: *Yes*, you can. There exist weights $\lambda_1, \dots, \lambda_k$, *not all zero*, such that

$$\lambda_1 \mathbf{x}_1 + \dots + \lambda_k \mathbf{x}_k = \mathbf{0}.$$

This means at least one of the vectors is “redundant” — it could have been expressed using the others. The set is wasting space.

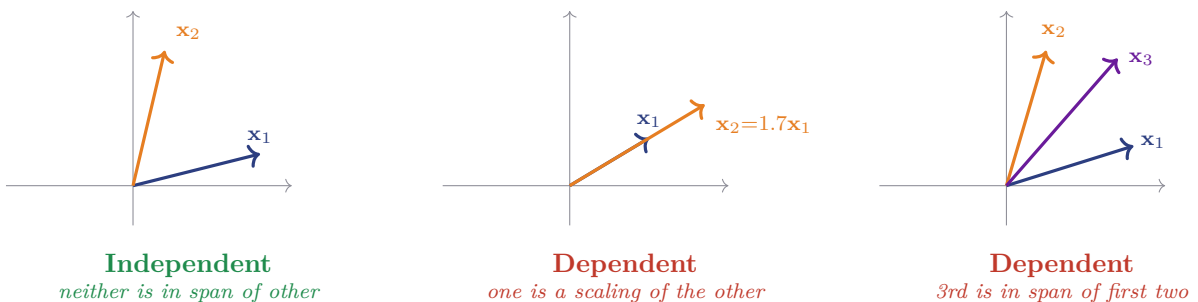
Linearly independent: *No*, it’s impossible. The *only* way to make them sum to $\mathbf{0}$ is the boring way: set every weight to zero. Each vector contributes something the others can’t replicate.

The plain-English version

Independent = “each vector brings something new to the table that none of the others can replicate.”

Dependent = “one of these vectors can already be expressed using the others — it’s redundant.”

6.1 The dependency-detector picture



Reading the picture, panel by panel:

- **Left.** The two arrows point in genuinely different directions. Neither is a scaled copy of the other. **Independent.**
- **Middle.** The second arrow is just the first one stretched (here, by a factor of 1.7). One is a scaling of the other — pure duplication. **Dependent.**
- **Right.** Three arrows in 2-D space. The third one isn’t a scaling of either of the first two, but it *can* be obtained as a mix of them (e.g. “a bit of the first, plus a bit of the second”). It’s redundant. **Dependent.**

Two handy shortcuts

- **The zero vector ruins independence instantly.** If $\mathbf{0}$ is in your set, the set is automatically dependent. (Just put weight 1 on $\mathbf{0}$ and 0 on everyone else — you’ve got a non-trivial combination summing to $\mathbf{0}$.)
- **For non-zero vectors:** dependent means *at least one of them is a mix of the others*. Independent means *no vector can be reconstructed from the rest*.

6.2 The algorithmic test: Gaussian elimination

You don't have to guess. There is a foolproof procedure.

The Gaussian elimination recipe

1. Stack the vectors as the **columns** of a matrix A .
2. Reduce A to row-echelon form.
3. Look at the columns:
 - **All columns have a pivot** $\Rightarrow$ vectors are **independent**.
 - **Some column has no pivot** $\Rightarrow$ vectors are **dependent**; in fact each non-pivot column is a linear combination of the pivot columns to its left.

A small example told step-by-step

We're handed three column vectors as a matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 4 \end{bmatrix}.$$

Question: are these three columns independent?

Step 1 — row-reduce. Subtract $2 \times$ (row 1) from row 2:

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 4 \end{bmatrix} \xrightarrow{R_2 - 2R_1} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & -2 \end{bmatrix}.$$

Step 2 — look at the pivots. Reading left to right, the leading entries (pivots) appear in **column 1** and **column 3**. **Column 2 has no pivot** — so it's redundant.

Step 3 — name the redundancy. Look back at column 2 of the original matrix: $\begin{bmatrix} 2 \\ 4 \end{bmatrix}$. It's just twice column 1: $2 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. So column 2 is a copy-with-scaling of column 1.

Verdict: the three columns are linearly **dependent**.

A bigger example with full narration

The setup. We have three vectors living in $\mathbb{R}^4$ (each is a column of 4 real numbers):

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 2 \\ -3 \\ 4 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 2 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} -1 \\ -2 \\ 1 \\ 1 \end{bmatrix}.$$

We want to know: are any of them redundant copies of the others?

Stack & reduce. Put them as columns of a single matrix and row-reduce:

$$\begin{bmatrix} 1 & 1 & -1 \\ 2 & 1 & -2 \\ -3 & 0 & 1 \\ 4 & 2 & 1 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 1 & 1 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

Read the pivots. Pivots show up in *all three* columns (column 1: the 1 at the top; column 2: the 1 in the second row; column 3: the 1 in the third row). No column is missing its pivot.

Verdict: $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ are linearly **independent** — none of them is built from the other two. The only way to mix them and get $\mathbf{0}$ is to set all weights to 0.

Why ML practitioners chase “independence”

“Linearly dependent features” is just the math name for what data scientists call *multicollinearity*.

- Two columns of your design matrix are nearly proportional $\Rightarrow$ the model has a (near-)null direction it can’t pin down. Coefficients become unstable and high-variance.
- Solutions in practice: drop the redundant feature, regularize (ridge/L2), or use PCA to project onto an independent set of directions.
- In deep learning, “filter redundancy” inside a CNN layer is the same idea — different filters learning essentially the same feature is wasted capacity.

7. The Independence-of-Combinations Theorem

This is the central technical idea of the lecture. It looks abstract at first, but it's the engine behind “finding a basis” and several proofs you'll see later in MFML.

7.1 The setup, told as a story

Imagine you already have k *trusted, non-redundant* building blocks: vectors $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_k$ that are linearly independent. From them, you build m new vectors $\mathbf{x}_1, \dots, \mathbf{x}_m$ — each one a recipe of how much of each $\mathbf{b}_i$ to mix:

$$\mathbf{x}_j = \lambda_{1j}\mathbf{b}_1 + \lambda_{2j}\mathbf{b}_2 + \dots + \lambda_{kj}\mathbf{b}_k.$$

Each new vector $\mathbf{x}_j$ is fully described by its *recipe*: the list of weights $(\lambda_{1j}, \dots, \lambda_{kj})$.

The natural question: are these new $\mathbf{x}_j$'s themselves independent, or did we accidentally build redundant ones?

7.2 Putting all the recipes into one picture

Pack the building blocks side-by-side as columns of a matrix:

$$B = [\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_k] \quad (\text{a matrix with the } \mathbf{b}_i \text{ as its columns}).$$

Pack the recipe for $\mathbf{x}_j$ into a column $\boldsymbol{\lambda}_j$. With these two pieces, building $\mathbf{x}_j$ becomes one matrix-vector product:

$$\mathbf{x}_j = B \boldsymbol{\lambda}_j.$$

Read it like this: “stack the building blocks in B , multiply by the recipe $\boldsymbol{\lambda}_j$, and out pops the new vector $\mathbf{x}_j$.”

$$\boxed{B \text{ (} k \text{ cols)}} \cdot \boxed{\boldsymbol{\lambda}_j} = \boxed{\mathbf{x}_j}$$

independent basis coordinates new vector

7.3 The theorem, in one sentence

The big result

The new vectors $\mathbf{x}_1, \dots, \mathbf{x}_m$ are independent if and only if their recipes $\boldsymbol{\lambda}_1, \dots, \boldsymbol{\lambda}_m$ are independent.

In symbols:

$$\{\mathbf{x}_1, \dots, \mathbf{x}_m\} \text{ independent} \iff \{\boldsymbol{\lambda}_1, \dots, \boldsymbol{\lambda}_m\} \text{ independent}.$$

What this is really saying: once you've fixed a trustworthy basis, checking independence in the original space is the *same problem* as checking independence among the recipes. It's a translation of the question into easier coordinates.

7.4 Why is this true? (a 2-step sketch)

We want to know whether the only way to make $\psi_1\mathbf{x}_1 + \dots + \psi_m\mathbf{x}_m = \mathbf{0}$ is to set every $\psi_j = 0$. Two steps:

Step 1 — substitute the recipes. Replace each $\mathbf{x}_j$ with $B \boldsymbol{\lambda}_j$:

$$\psi_1\mathbf{x}_1 + \dots + \psi_m\mathbf{x}_m = \psi_1 B \boldsymbol{\lambda}_1 + \dots + \psi_m B \boldsymbol{\lambda}_m = B (\psi_1 \boldsymbol{\lambda}_1 + \dots + \psi_m \boldsymbol{\lambda}_m).$$

Call the bracketed expression $\mathbf{v}$.

Step 2 — use the trustworthy basis. Since the columns of B (our independent $\mathbf{b}_i$) are independent, the only way $B\mathbf{v}$ can equal $\mathbf{0}$ is if $\mathbf{v} = \mathbf{0}$. In other words, the original question

$$\psi_1 \mathbf{x}_1 + \cdots + \psi_m \mathbf{x}_m = \mathbf{0}$$

is exactly the same as the recipe-level question

$$\psi_1 \boldsymbol{\lambda}_1 + \cdots + \psi_m \boldsymbol{\lambda}_m = \mathbf{0}.$$

And that's exactly the linear-independence question for the recipes. ■

The “coordinates inherit independence” picture

Once you fix an independent set $\mathbf{b}_1, \dots, \mathbf{b}_k$, every new vector built from them is fully described by its coordinate vector $\boldsymbol{\lambda}_j$. **The independence/dependence of the new vectors is the same as the independence/dependence of their coordinate vectors.** Geometry in the original space mirrors geometry in coordinate space — that mirror is the whole reason coordinate systems are useful in the first place.

7.5 The big consequence: too many vectors $\Rightarrow$ guaranteed redundancy

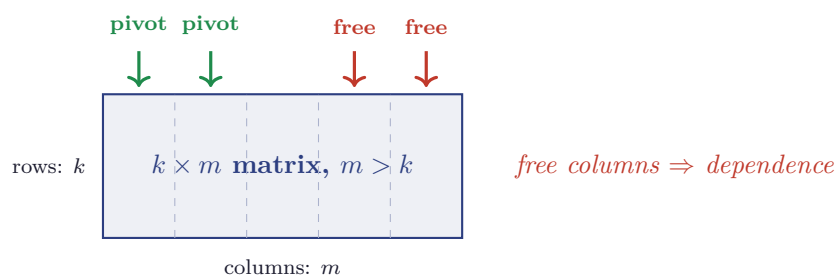
Each recipe $\boldsymbol{\lambda}_j$ is a column with k entries (one weight per building block). So the recipes live inside $\mathbb{R}^k$ — a space “ k entries wide.”

If you build *more* new vectors than you have building blocks ($m > k$), you're forcing more recipes than $\mathbb{R}^k$ can hold independently. The recipes *must* have a hidden relation among them, and by the theorem so do the $\mathbf{x}_j$.

Pigeonhole consequence — in plain English

If you build more new vectors than you have independent building blocks ($m > k$), the new vectors are guaranteed to be redundant.

The matrix view: stacking the recipes as columns gives a $k \times m$ matrix with more columns than rows. That matrix can have at most k pivots, so there must be free (non-pivot) columns — which is exactly what dependence looks like.



7.6 Worked example, told as a story

The setup. Imagine someone hands us four “trustworthy” building-block vectors $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3, \mathbf{b}_4$, and they're already known to be independent — think of them like four basic ingredients in your pantry. We don't even need to know what they are concretely.

Now we use them to build four new vectors via the following recipes:

$$\begin{aligned} \mathbf{x}_1 &= 1\mathbf{b}_1 - 2\mathbf{b}_2 + 1\mathbf{b}_3 - 1\mathbf{b}_4 \\ \mathbf{x}_2 &= -4\mathbf{b}_1 - 2\mathbf{b}_2 + 0\mathbf{b}_3 + 4\mathbf{b}_4 \\ \mathbf{x}_3 &= 2\mathbf{b}_1 + 3\mathbf{b}_2 - 1\mathbf{b}_3 - 3\mathbf{b}_4 \\ \mathbf{x}_4 &= 17\mathbf{b}_1 - 10\mathbf{b}_2 + 11\mathbf{b}_3 + 1\mathbf{b}_4 \end{aligned}$$

The question. Are these four new “dishes” $\mathbf{x}_1, \dots, \mathbf{x}_4$ independent, or did we accidentally cook the same dish twice (under different names)?

Step 1 — write down each recipe as a column. The recipe for $\mathbf{x}_j$ is just the list of weights it uses — one weight per ingredient. Stack the four recipes side-by-side:

$$A = \begin{bmatrix} 1 & -4 & 2 & 17 \\ -2 & -2 & 3 & -10 \\ 1 & 0 & -1 & 11 \\ -1 & 4 & -3 & 1 \end{bmatrix}.$$

Each column is one recipe; each row is the weight on a specific ingredient.

Step 2 — row-reduce. (The mechanics from Section 6.) After elimination, the matrix simplifies to:

$$A \rightsquigarrow \begin{bmatrix} 1 & 0 & 0 & -7 \\ 0 & 1 & 0 & -15 \\ 0 & 0 & 1 & -18 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Step 3 — read the pivots. Columns 1, 2, 3 have pivots. **Column 4 doesn’t.** The non-pivot column tells us the exact recipe-redundancy: $\mathbf{x}_4$ ’s recipe is $7\mathbf{x}_1$ ’s + $15\mathbf{x}_2$ ’s + $18\mathbf{x}_3$ ’s. In symbols:

$$7\lambda_1 + 15\lambda_2 + 18\lambda_3 = \lambda_4.$$

Step 4 — translate back via the theorem. By the theorem, the recipes’ relation transfers to the dishes:

$$7\mathbf{x}_1 + 15\mathbf{x}_2 + 18\mathbf{x}_3 - \mathbf{x}_4 = \mathbf{0}.$$

Verdict: the four dishes $\mathbf{x}_1, \dots, \mathbf{x}_4$ are linearly **dependent**, and we even spotted the *exact* redundancy: $\mathbf{x}_4$ is a particular weighted sum of the first three.

Why this theorem matters in ML

- **Dimensionality reduction.** Once you choose a small basis $\mathbf{b}_1, \dots, \mathbf{b}_k$ (e.g., principal components), every data point is just its coordinate vector in $\mathbb{R}^k$. ML on the coordinates is mathematically equivalent to ML on the originals — the theorem above is what guarantees this.
- **Embedding analyses.** When you ask “are these 50 word vectors independent in a 32-dim space?” the answer is *always no* ($50 > 32$) — you’ve discovered a relation purely from dimension counting.
- **Capacity arguments.** A neural network layer with k neurons can only produce activations in a k -dimensional subspace. Any $m > k$ output activations across a batch must satisfy a linear relation.

8. Basis and Dimension: The Skeleton of a Space

We’ve seen that span tells us *what we can build*, and independence tells us *when our building blocks are non-redundant*. A basis is the perfect marriage of both.

A grounded story to anchor everything

Imagine you’re at the origin of a flat 2-D map and you want to be able to reach *any* point on the map by walking. You can only walk in straight lines, and you’re allowed to combine walks (“walk 3 steps in direction A, then 2 steps in direction B”).

- If you only know *north*, you can reach the entire vertical axis but no east-west points. **Not enough directions** — you don’t have a generating set of the plane.
- If you know *north* and *east*, you can reach *any* point on the map. **Generating set!**
- If you know *north*, *east*, and *north-east*, you can still reach any point — but “north-east” is just a mix of the other two; it’s redundant. **Generating set, but not minimal.**
- If you know *north* and *east* (and only those), you have the just-right toolkit: *enough* to reach everything, with *nothing wasted*. That’s a **basis**.

The whole rest of this section is just formal vocabulary for the three situations above.

Generating Set and Basis — in plain English

Take a small collection of vectors $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$ in some vector space V :

- It’s a **generating set** of V if you can build *every* vector in V by mixing these with weights. (Like “north + east is enough to reach the whole 2-D plane.”)
- It’s a **basis** of V if it’s a generating set *and* no vector in it is redundant. (Like “north + east, with nothing extra.”)

One-sentence slogan. *A basis is the smallest possible toolkit that still reaches everything in the space.*

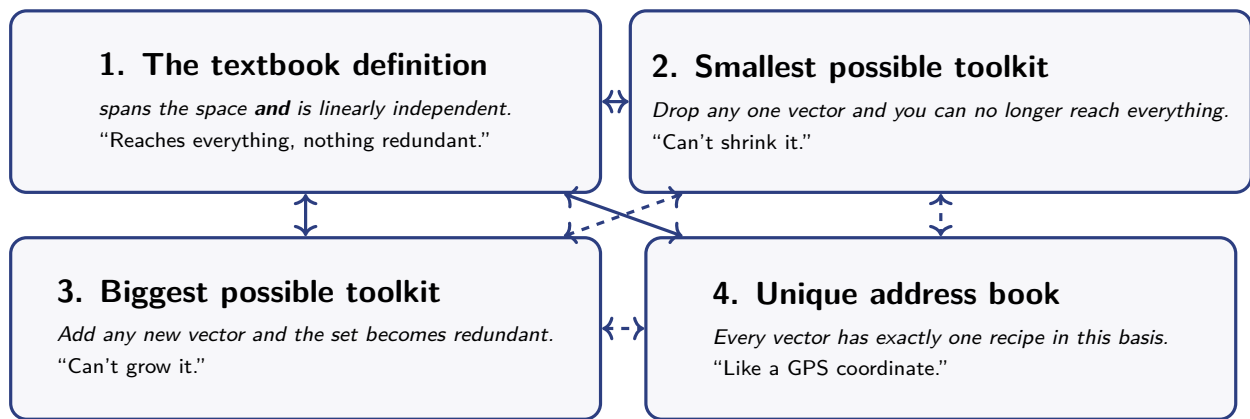
Recipe analogy — when does each name apply?

Think of building vectors as cooking dishes:

- **Not a generating set:** your pantry is missing ingredients — you literally cannot make some dishes.
- **Generating set, but not minimal:** you have everything you need, plus some duplicates (two bottles of soy sauce). You can cook everything, but you’ve wasted shelf space.
- **Basis:** every ingredient is essential, and together they let you make every dish. Remove any one and some dish becomes impossible to make.

8.1 Four ways to spot a basis

A basis can be described in four different-looking ways, but all four are saying the exact same thing. Each angle gives a different gut feeling. Use whichever feels easier in the moment.



Let’s walk through what each one really means with a concrete picture.

- **(1) Textbook definition.** Like “*north* and *east*” on a 2-D map: together they reach every point (spans), and neither is a copy of the other (independent). This is the formal version of the Goldilocks story.
- **(2) Smallest possible toolkit.** If you tried to remove “*east*,” you’d lose every east-west point. So you *can’t shrink* the toolkit. This view is helpful when you want to prove a basis exists — start with a generating set and toss out redundant vectors one at a time until you can’t anymore.
- **(3) Biggest possible toolkit.** If you tried to add “*north-east*” to the basis, it would already be expressible as “*north + east*” — redundant. So you *can’t grow* the toolkit while staying independent. This view is helpful when you want to extend a partial set into a basis — keep adding non-redundant vectors until you can’t anymore.
- **(4) Unique address book. This is the most useful one in practice.** Once you’ve fixed a basis, every vector has *exactly one* recipe in it. Like GPS coordinates: there’s exactly one (latitude, longitude) for the Eiffel Tower — not two, not three. Without this uniqueness, “coordinates” wouldn’t even be a meaningful idea.

Why all four are the same fact

Property (1) is just (2) and (3) said together: “spans” = “can’t shrink any further while still spanning” (that’s (2)), and “independent” = “can’t grow without losing independence” (that’s (3)). And (4) follows from independence: if the basis weren’t independent, the same vector could have two different recipes (we proved this on the previous page).

Why does a basis give unique coordinates?

Suppose a single vector $\mathbf{v}$ has *two* different recipes in the basis:

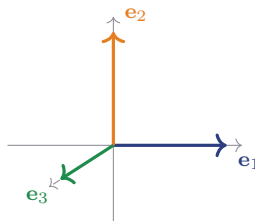
$$\mathbf{v} = \lambda_1 \mathbf{b}_1 + \cdots + \lambda_k \mathbf{b}_k = \psi_1 \mathbf{b}_1 + \cdots + \psi_k \mathbf{b}_k.$$

Subtracting, the difference recipe satisfies $(\lambda_1 - \psi_1) \mathbf{b}_1 + \cdots + (\lambda_k - \psi_k) \mathbf{b}_k = \mathbf{0}$. But the $\mathbf{b}_i$ are independent — so the only way that combination can be $\mathbf{0}$ is if every weight in it is zero. That forces $\lambda_i = \psi_i$ for each i .

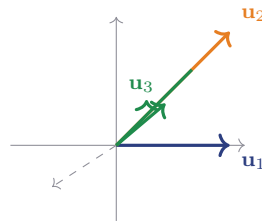
Moral: the recipe is forced to be unique, *because* the basis is independent. “Basis = unique addresses” is just another way of saying “no redundancy in the basis.”

8.2 A basis is not unique

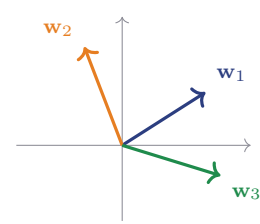
The same space $\mathbb{R}^3$ can be described by many different bases.



Canonical
(1,0,0), (0,1,0), (0,0,1)



Triangular
(1,0,0), (1,1,0), (1,1,1)



Generic / Rotated
any 3 indep. vectors

Why “different bases” is one of the most useful ideas in ML

- **PCA** = “find a new orthonormal basis for the data, ordered by variance.”
- **Fourier transform** = “re-express a signal in the basis of sines and cosines.”
- **Wavelets, ICA, autoencoders** = “find a basis where the structure of the data becomes obvious.”

The data hasn’t changed, the underlying space hasn’t changed — but with a smarter basis, the right algorithm becomes a one-line solution. Choosing the right basis *is* much of the art of representation learning.

8.3 Dimension

Dimension — in plain English

Pick *any* basis of a (finite-dimensional) vector space V and count how many vectors are in it. Pick a different basis and count again. **You’ll always get the same number.**

That number is intrinsic to the space itself — we call it the **dimension** of V , written $\dim(V)$. It’s the answer to “how many independent directions does this space really have?”

- $\dim(\mathbb{R}^n) = n$ — a column of n real numbers needs n basis vectors (the standard ones $\mathbf{e}_1, \dots, \mathbf{e}_n$).
- $\dim(\mathbb{R}^{m \times n}) = m \cdot n$ — an $m \times n$ matrix has mn entries; a basis is the family of “one-hot” matrices E_{ij} that have a single 1 at position (i, j) and 0 elsewhere.
- If $U \subseteq V$, then $\dim(U) \leq \dim(V)$, with equality iff $U = V$ (a subspace can never be “bigger” than the space it lives in).

A very common confusion

The number of components in a vector is not always the dimension of its space. Consider

$$V = \text{span}\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}\right).$$

Each vector has 2 components, but the space is just the y -axis: it’s a *line*, not a plane. So $\dim(V) = 1$. Components count the *ambient* space; dimension counts the *intrinsic* space.

Ambient vs. intrinsic dimension

A road on a 2-D map has only 1 dimension of freedom: “how far along the road.” The map (the ambient space) is 2-D, but the road (the subspace) is 1-D. ML calls this distinction *intrinsic dimensionality*, and a huge amount of representation learning is about discovering it: real-world data tends to live on a low-dimensional surface inside its high-dimensional ambient space — the famous *manifold hypothesis*.

9. Finding a Basis: The Algorithm

You're handed a bunch of vectors that span some subspace $U \subseteq \mathbb{R}^n$. How do you extract a basis from them?

The 3-step procedure

A basis of $U = \text{span}(\mathbf{x}_1, \dots, \mathbf{x}_m) \subseteq \mathbb{R}^n$ can be found by:

1. Stack the spanning vectors as columns of a matrix A .
2. Reduce A to row-echelon form.
3. The *original* spanning vectors corresponding to the *pivot columns* form a basis of U .

Important nuance

The basis is the **original** pivot columns of A , not the columns of the row-reduced matrix. Row reduction tells you *which* columns to keep, not *what they are*.

9.1 Worked example: a basis for a subspace of $\mathbb{R}^5$ — told step by step

The setup. We're handed four vectors, each living in $\mathbb{R}^5$ (i.e., each is a column of 5 real numbers). They span some subspace U inside $\mathbb{R}^5$ — but we don't know yet how “big” U really is, or whether some of these four are redundant.

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 2 \\ -1 \\ -1 \\ -1 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 2 \\ -1 \\ 1 \\ 2 \\ -2 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} 3 \\ -4 \\ 3 \\ 5 \\ -3 \end{bmatrix}, \quad \mathbf{x}_4 = \begin{bmatrix} -1 \\ 8 \\ -5 \\ -6 \\ 1 \end{bmatrix}.$$

The goal. Find the smallest sub-collection of these four that still spans the same subspace — in other words, throw out the redundant ones and keep a basis.

Step 1 — stack and reduce. Put them as columns of a single 5×4 matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 & -1 \\ 2 & -1 & -4 & 8 \\ -1 & 1 & 3 & -5 \\ -1 & 2 & 5 & -6 \\ -1 & -2 & -3 & 1 \end{bmatrix}.$$

Row-reduce: kill everything below the first 1 in column 1, then move to column 2 and do the same, and so on. The reduced form comes out to:

$$\text{rref}(A) = \begin{bmatrix} 1 & 0 & -1 & 3 \\ 0 & 1 & 2 & -2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Step 2 — spot the pivots. The leading 1's sit in **columns 1 and 2**. **Columns 3 and 4 have no pivots** — meaning vectors $\mathbf{x}_3$ and $\mathbf{x}_4$ are redundant copies in disguise.

Step 3 — read off the redundancies. The reduced form literally tells us the recipes:

- Column 3 says: $\mathbf{x}_3 = -1 \cdot \mathbf{x}_1 + 2 \cdot \mathbf{x}_2$. (Read down column 3 of rref: -1 and 2 .)
- Column 4 says: $\mathbf{x}_4 = 3 \cdot \mathbf{x}_1 - 2 \cdot \mathbf{x}_2$. (Read down column 4 of rref: 3 and -2 .)

So $\mathbf{x}_3$ and $\mathbf{x}_4$ are nothing more than mixes of $\mathbf{x}_1$ and $\mathbf{x}_2$.

The Result

A basis of U is $\{\mathbf{x}_1, \mathbf{x}_2\}$. Since the basis has 2 vectors, $\dim(U) = 2$.

Geometric punchline. The subspace U *lives inside* $\mathbb{R}^5$ (a 5-dimensional space) but is itself only *2-dimensional* — it’s a flat plane sitting inside $\mathbb{R}^5$, passing through the origin. The original 4 vectors all lay on this single plane, but the plane really only needs 2 directions to describe.

9.2 Why this procedure works (1-paragraph proof sketch)

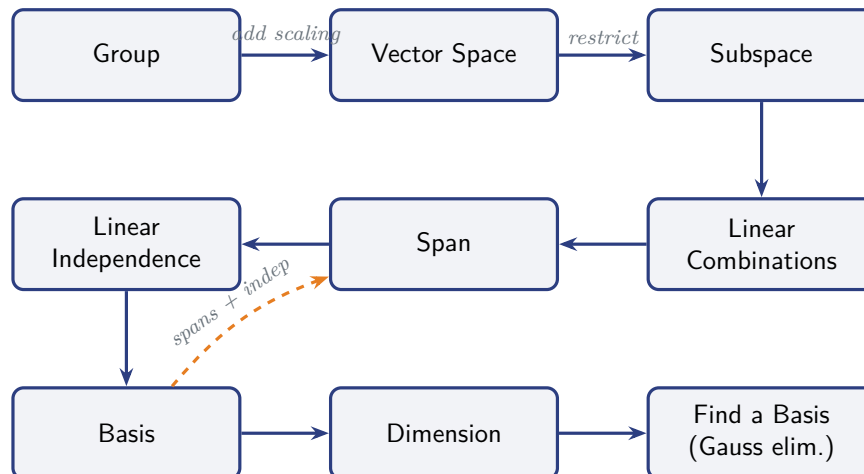
Row operations don’t change which subset of column vectors is independent — they only change the *rows* (the way each column vector “looks”). So the columns that remain independent in the reduced form are precisely the columns that were independent in the original. By the theorem of Section 7, the pivot columns of A are independent; the non-pivot columns are linear combinations of pivot columns to their left. Hence the original pivot columns span U and are independent — a basis.

Connection to model capacity

The number of pivot columns is the *rank* of the matrix A — the dimension of its column space. In ML:

- For a feature matrix X , $\text{rank}(X)$ tells you how many of your features are truly independent. Anything beyond that is collinearity.
- For a neural network weight matrix W , $\text{rank}(W)$ caps how many “effective directions” the layer can express. Low-rank approximations (LoRA, factorized layers) explicitly exploit this to compress models.
- For a Jacobian J of a network, $\text{rank}(J)$ describes how many directions of input change actually affect the output.

10. Putting It All Together



10.1 The ML cheat sheet for Lecture 2

Concept	One-line meaning	Where it shows up in ML/DS
Vector space	“Universe” closed under $+$ and $\cdot$	Inputs, outputs, weights, gradients all live here.
Subspace	A flat slice through $\mathbf{0}$	Column space, null space, low-rank manifolds, PCA subspace.
Span	Set of all linear combinations	Hypothesis class for linear models.
Linear independence	No redundancy among vectors	Multicollinearity check; independent features.
Basis	Smallest spanning set	Coordinate system; PCA components, Fourier basis.
Dimension	Size of any basis	Intrinsic dimensionality of data; layer capacity.
Rank (preview)	# pivots = dim of column space	Rank of X , low-rank approx., LoRA, model capacity.

10.2 The mental loop you should run from now on

Four questions to ask whenever you see a vector or matrix in ML

1. **What space does this live in?** (column space? null space? embedding space?)
2. **Is this set of vectors independent, or redundant?** (collinear features? duplicate filters?)
3. **What is the span / column space / null space / rank?** (capacity, expressiveness, blind spots.)
4. **Is there a smarter basis?** (PCA, Fourier, wavelets, learned embeddings.)

This four-question loop is most of the geometric reasoning behind classical ML and a great deal of modern deep learning.

11. Practice Problems

Problem 1 — Subspace test

Which of the following are subspaces of $\mathbb{R}^3$?

- (a) $\{(x, y, z) : x + y + z = 0\}$
- (b) $\{(x, y, z) : x + y + z = 1\}$
- (c) $\{(x, y, z) : x \geq 0\}$
- (d) $\{(x, y, z) : x = 2y\}$

Hint. Apply the three subspace tests. Anything that fails closure under scaling (e.g., negative scalars) or doesn't contain $\mathbf{0}$ is out.

Problem 2 — Express as a linear combination

Write

$$\mathbf{b} = \begin{bmatrix} 4 \\ 3 \\ 1 \\ 2 \end{bmatrix} \text{ as a linear combination of } \mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 1 \end{bmatrix}, \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ -2 \\ 2 \end{bmatrix}, \mathbf{v}_3 = \begin{bmatrix} 3 \\ -1 \\ 0 \\ 1 \end{bmatrix}, \mathbf{v}_4 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

Hint. Set up the augmented system $[\mathbf{v}_1 \ \mathbf{v}_2 \ \mathbf{v}_3 \ \mathbf{v}_4 \mid \mathbf{b}]$ and row-reduce.

Problem 3 — Independence theorem in action

Let $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$ be linearly independent. Define

$$\mathbf{x}_1 = \mathbf{b}_1 + \mathbf{b}_2, \quad \mathbf{x}_2 = \mathbf{b}_2 + \mathbf{b}_3, \quad \mathbf{x}_3 = \mathbf{b}_1 + \mathbf{b}_3.$$

Are $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ independent? Use the theorem from Section 7 (do not work in the $\mathbf{b}_i$ explicitly).

Problem 4 — Is it a basis?

Consider $S = \{(2, 3, 5), (5, 7, 9), (1, 11, 1)\}$ in $\mathbb{R}^3$. Is S a basis? Justify in two ways: (a) using row reduction, (b) using the dimension argument (any 3 independent vectors in a 3-D space form a basis).

Problem 5 — Find a basis

Find a basis (and the dimension) of the subspace of $\mathbb{R}^4$ spanned by

$$\begin{bmatrix} 1 \\ 2 \\ 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 4 \\ 2 \\ 6 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 3 \\ 2 \\ 3 \\ 5 \end{bmatrix}.$$

Notice that two of these vectors are obvious scalings/sums of the others — the row reduction will confirm it.

*Vector spaces are the stage. Linear independence is the casting. A basis is the cast.
The rest of MFML is the play. Enjoy the show.*